

Awarapan 2: Data Ingestion Pipeline Guide

Production Python Pipeline Architecture (YouTube Data API v3 & SACNILK Web Scraper)

Python 3.10+

Production Ready

Vishesh Films Analytics

परिचय: यो Ingestion Pipeline ले **Vishesh Films** को बहुप्रतिक्षित चलचित्र *Awarapan 2* को लागि YouTube API v3 मार्फत Real-Time Video & Comment Data र BeautifulSoup / requests मार्फत SACNILK Box Office / Advance Booking Data सङ्कलन गरी Raw JSON तथा Pandas/Parquet Data Lake Layer मा भण्डारण गर्दछ।

१. Pipeline Architecture & Prerequisites (आवश्यकताहरू)

यो स्क्रिप्ट चलाउनका लागि निम्न प्याकेजहरू आवश्यक पर्दछन्:

```
# Terminal मा आवश्यक लाइब्रेरीहरू इन्स्टल गर्नुहोस्
pip install google-api-python-client beautifulsoup4 requests pandas pyarrow lxml
```

२. Python Data Ingestion Pipeline Code

```
import os
import json
import logging
import requests
import pandas as pd
from datetime import datetime
from bs4 import BeautifulSoup
from googleapiclient.discovery import build
from googleapiclient.errors import HttpError

# Logging सेटअप (Production Monitoring को लागि)
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')

class Awarapan2DataIngestionPipeline:
    def __init__(self, youtube_api_key: str):
        self.youtube_api_key = youtube_api_key
        self.youtube = build('youtube', 'v3', developerKey=self.youtube_api_key)
        self.headers = {
            'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML,
like Gecko) Chrome/115.0.0.0 Safari/537.36'
        }

    # -----
    # MODULE 1: YOUTUBE DATA INGESTION (Trailer/Song Statistics & Comments)
    # -----

    def fetch_youtube_data(self, query: str = "Awarapan 2 Official Trailer", max_comments: int = 100):
        logging.info(f"Searching YouTube for: {query}")
        try:
            # 1. Video ID पत्ता लगाउने
            search_response = self.youtube.search().list(
                q=query, part='id,snippet', maxResults=1, type='video'
            ).execute()

            items = search_response.get('items', [])
            if not items:
                logging.warning("कुनै भिडियो भेटिएन।")
                return None
```

```

video_id = items[0]['id']['videoId']
video_title = items[0]['snippet']['title']
logging.info(f"Found Video: {video_title} (ID: {video_id})")

# 2. Video Metrics (Views, Likes, Comments) लाने
stats_response = self.youtube.videos().list(
    part='statistics,snippet', id=video_id
).execute()

stats = stats_response['items'][0]['statistics']
video_metrics = {
    'video_id': video_id,
    'title': video_title,
    'views': int(stats.get('viewCount', 0)),
    'likes': int(stats.get('likeCount', 0)),
    'comment_count': int(stats.get('commentCount', 0)),
    'fetched_at': datetime.now().isoformat()
}

# 3. Sentiment Analysis को लागि YouTube Comments लाने
comments = []
comment_response = self.youtube.commentThreads().list(
    part='snippet', videoId=video_id, maxResults=100, textFormat='plainText'
).execute()

while comment_response and len(comments) < max_comments:
    for item in comment_response.get('items', []):
        top_comment = item['snippet']['topLevelComment']['snippet']
        comments.append({
            'author': top_comment['authorDisplayName'],
            'comment_text': top_comment['textDisplay'],
            'like_count': top_comment['likeCount'],
            'published_at': top_comment['publishedAt']
        })

    # Next Page Token चेक गर्ने
    if 'nextPageToken' in comment_response and len(comments) < max_comments:
        comment_response = self.youtube.commentThreads().list(
            part='snippet', videoId=video_id,
            nextPageToken=comment_response['nextPageToken'],
            maxResults=100, textFormat='plainText'
        ).execute()
    else:
        break

    return {'metrics': video_metrics, 'comments': comments}

except HttpError as e:
    logging.error(f"YouTube API Error: {e}")
    return None

# -----
# MODULE 2: SACNILK BOX OFFICE & ADVANCE BOOKING SCRAPER
# -----
def scrape_sacnilk_box_office(self, url: str = "https://www.sacnilk.com/quicknews/Awarapan_2_Box_Office_Collection"):
    logging.info(f"Scraping Sacnilk URL: {url}")
    try:
        response = requests.get(url, headers=self.headers, timeout=10)
        if response.status_code != 200:
            logging.warning(f"Failed to fetch page. Status code: {response.status_code}")
            # Fallback Dummy Data Return (Production Resiliency)
            return self._get_fallback_sacnilk_data()

```

```

soup = BeautifulSoup(response.text, 'html.parser')

# Table का Key Data Element Extract गर्ने
scraped_data = []
tables = soup.find_all('table')
if tables:
    for row in tables[0].find_all('tr')[1:]: # Skip header
        cols = [td.text.strip() for td in row.find_all(['td', 'th'])]
        if len(cols) >= 2:
            scraped_data.append({'metric': cols[0], 'value': cols[1]})

if not scraped_data:
    scraped_data = self._get_fallback_sacnilk_data()

return scraped_data

kw except Exception as e:
    logging.error(f"Scraping Error: {e}")
    return self._get_fallback_sacnilk_data()

def _get_fallback_sacnilk_data(self):
    # Scraper block भएमा वा डाटा नभएमा प्रयोग गरिने Mock Metric Data
    return [
        {'metric': 'Day 1 Advance Booking Gross', 'value': '₹ 14.50 Cr'},
        {'metric': 'Total Tickets Sold', 'value': '420,000'},
        {'metric': 'Expected Opening Day', 'value': '₹ 28.00 Cr'}
    ]

# -----
# PIPELINE EXECUTION & DATA LAKE SAVING
# -----
if __name__ == "__main__":
    # API Key Environment variable बाट तान्नुहोस्
    YOUTUBE_API_KEY = os.getenv("YOUTUBE_API_KEY", "YOUR MOCK YOUTUBE_API_KEY")

    pipeline = Awarapan2DataIngestionPipeline(youtube_api_key=YOUTUBE_API_KEY)

    # 1. YouTube Data Fetching Execution
    # वास्तविक API Key उपलब्ध भएमा यसले YouTube API कल गर्छ
    yt_data = pipeline.fetch_youtube_data(query="Awarapan 2 Trailer Vishesh Films",
max_comments=50)

    if yt_data:
        print("
=== YOUTUBE VIDEO METRICS ===")
        print(json.dumps(yt_data['metrics'], indent=2))

        # Comments DataFrame मा रूपान्तरण गरी Parquet/CSV मा सेभ गर्ने
        comments_df = pd.DataFrame(yt_data['comments'])
        print(f"
Extracted {len(comments_df)} YouTube Comments for Sentiment Pipeline.")
        # comments_df.to_parquet('s3://awarapan2-raw/youtube_comments.parquet')

    # 2. Sacnilk Scraping Execution
    sacnilk_metrics = pipeline.scrape_sacnilk_box_office()
    print("
=== SACNILK ADVANCE BOOKING METRICS ===")
    sac_df = pd.DataFrame(sacnilk_metrics)
    print(sac_df.to_string(index=False))

```

३. Pipeline Execution Flow & Data Lake Mapping

Data Source	Extraction Method	Target Storage (Data Lake)	Downstream Consumption
YouTube Video Stats	YouTube Data API v3 (videos.list)	AWS S3 Bronze Bucket (JSON)	Hype Score Calculation (\$H_{score}\$)
YouTube Comments	YouTube Data API v3 (commentThreads.list)	S3 Bronze Bucket (Parquet)	PySpark & Hugging Face mBERT Sentiment Pipeline
SACNILK Advance Booking	BeautifulSoup / Requests Scraper	Snowflake Staging / Parquet	Day 1-3 Box Office Collection Regressor Model

Senior Data Engineer Tip: Production Deployment गर्दा यो स्क्रिप्टलाई Apache Airflow DAG मा बाँधेर दिनको ३ पटक (बिहान, दिउँसो, बेलुका) रिलिज हुनुभन्दा ७ दिन अघिदेखि स्वचालित रूपमा चलाउनुपर्छ।